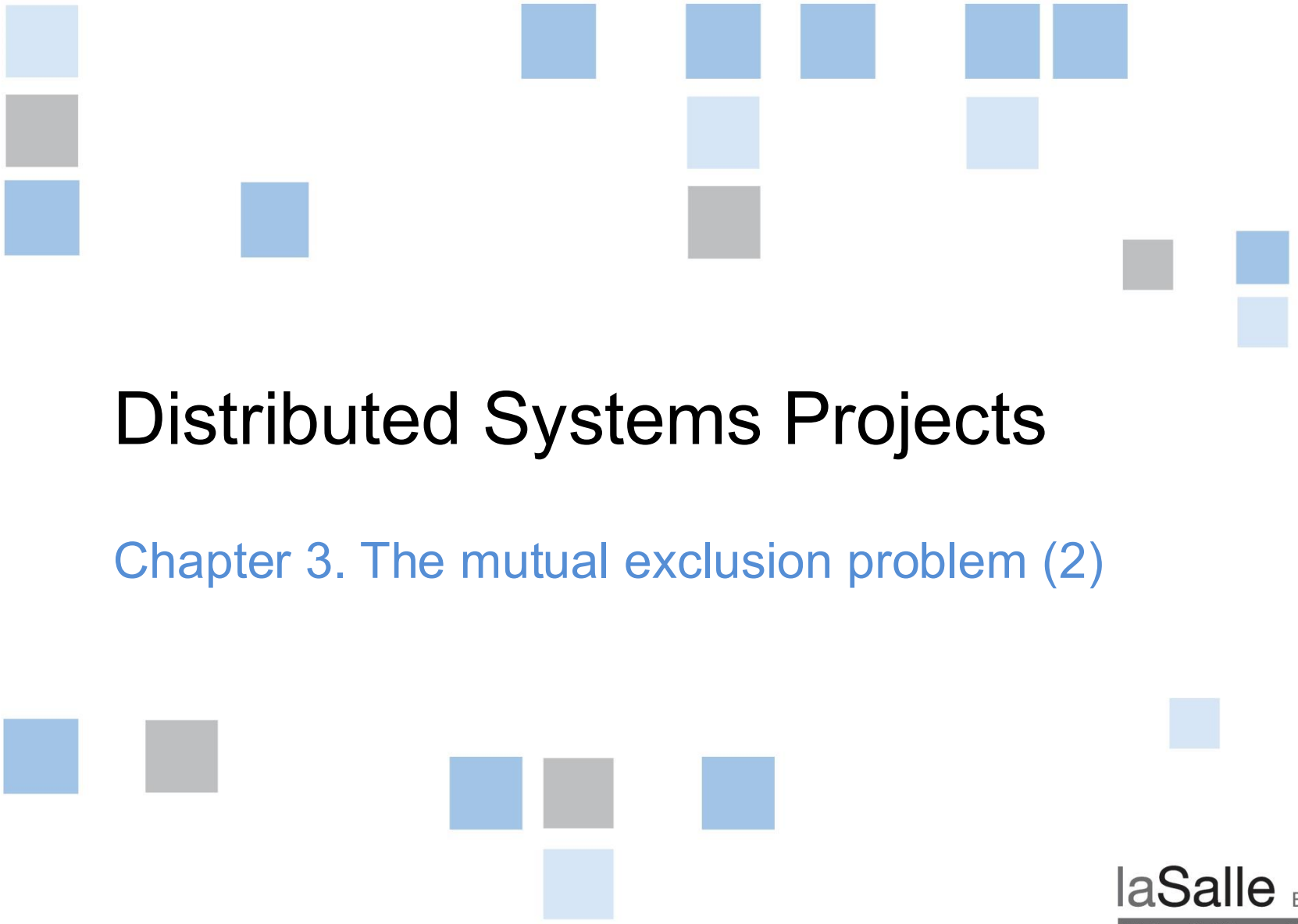


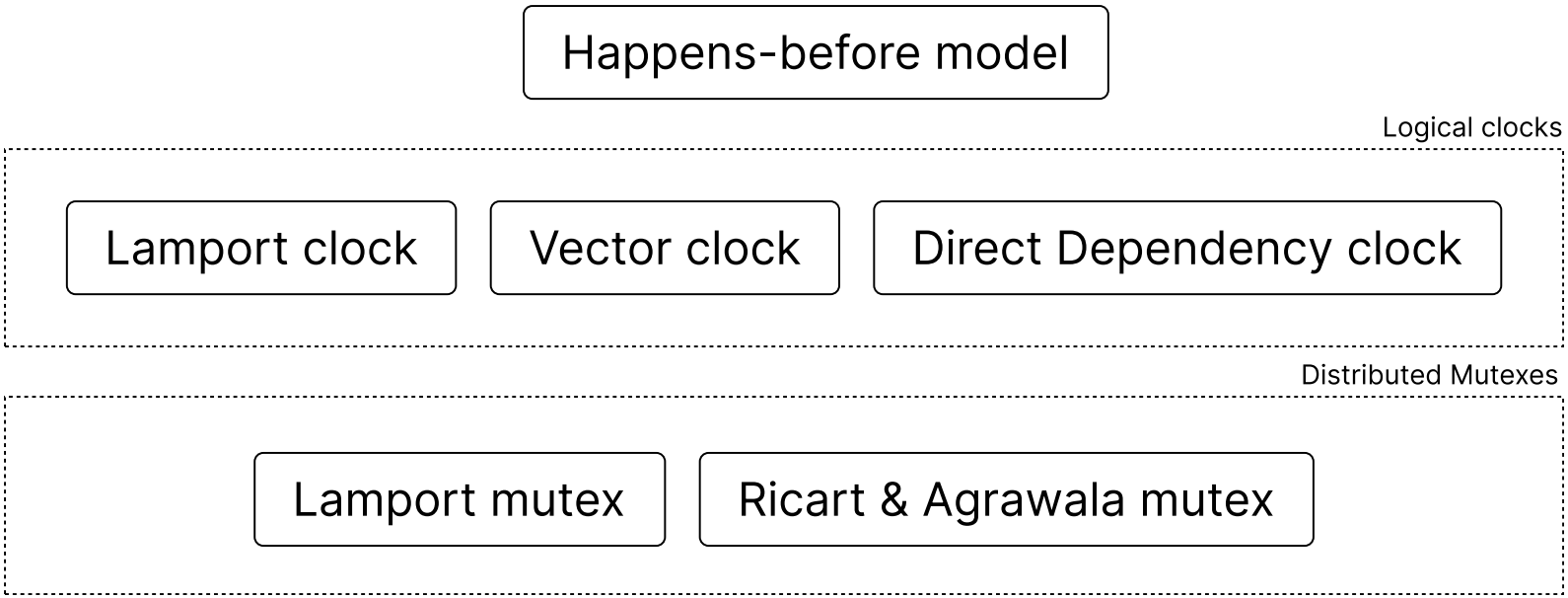
Distributed Systems Projects

Chapter 3. The mutual exclusion problem (2)

laSalle ENG
Universitat Ramon Llull

Today's agenda:

- Mutual exclusion definition
- Centralized algorithm
- Lamport algorithm
- Ricart & Agrawala algorithm


Happens-before model
Logical clocks
Lamport clock
Vector clock
Direct Dependency clock
Distributed Mutexes
Lamport mutex
Ricart & Agrawala mutex

Ricart-Agrawala's algorithm

Ricart-Agrawala has an (n - 1) improvement over Lamport's algorithm.

This approach requires 2(n - 1) messages.

Message types of the algorithm

1. **Request**: Request petition when a node wants to access the Critical Section.
2. ~~**Acknowledgement**: Message sent back to the requesting node to give permission to enter the CS.~~
3. **Reply (New ACK)**: A message that is sent back to the requester.

Ricart-Agrawala's algorithm

Steps of the algorithm

Requester

1. Sends a REQUEST message to all other nodes and expects a REPLY message back.
2. On CS exit, it sends a REPLY message to all deferred REQUESTs in the queue.

Other nodes

1. Not in the CS and doesn't want to access it, sends REPLY back.
2. In the CS, enqueues the petition (to be processed on exit)
3. Not in the CS, but wants to access it, and the requester timestamp is lower, REPLY.
4. Not in the CS, but wants to access it, and the requester timestamp is higher, enqueue the petition.
5. Not in the CS, but wants to access it, and the requester has the same timestamp, tie break by ID, otherwise, enqueue the petition.

Ricart-Agrawala's algorithm

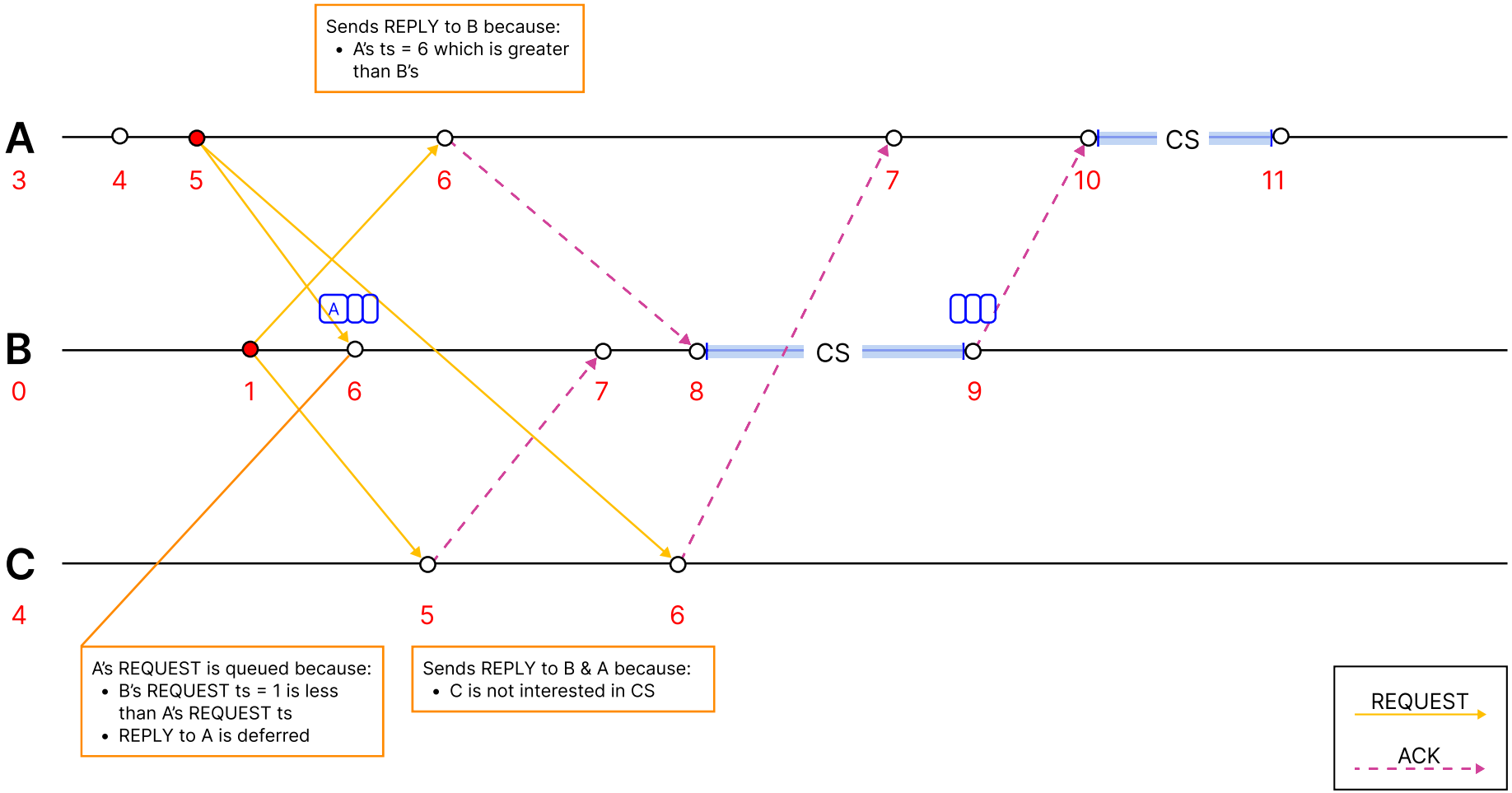
A process can access the CS iff satisfies the following conditions:

Requesting node PoW

1. It has requested the CS.
2. Received all REPLY (New ACK) messages from all other nodes.

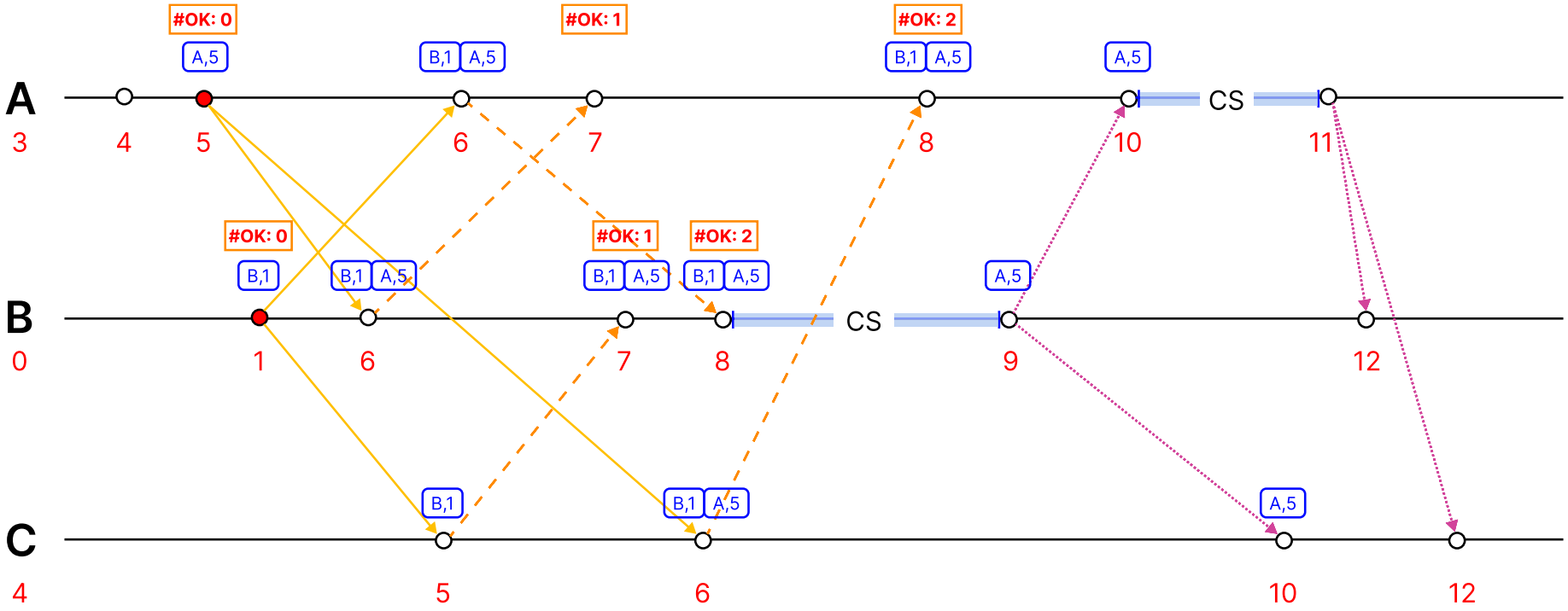
Ricart-Agrawala's distributed mutex Algorithm

Ricart-Agrawala Distributed Mutual Exclusion with Lamport clock



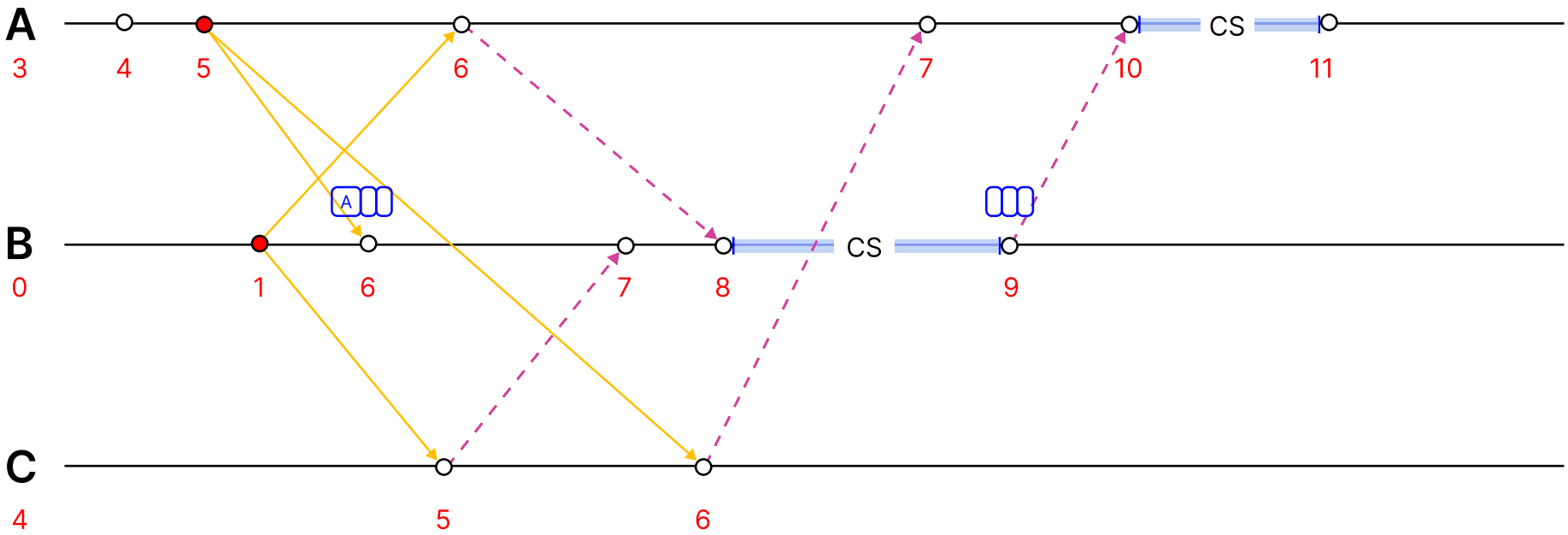
Lamport / Ricart-Agrawala – A comparison side by side

Lamport Distributed Mutual Exclusion with Lamport clock

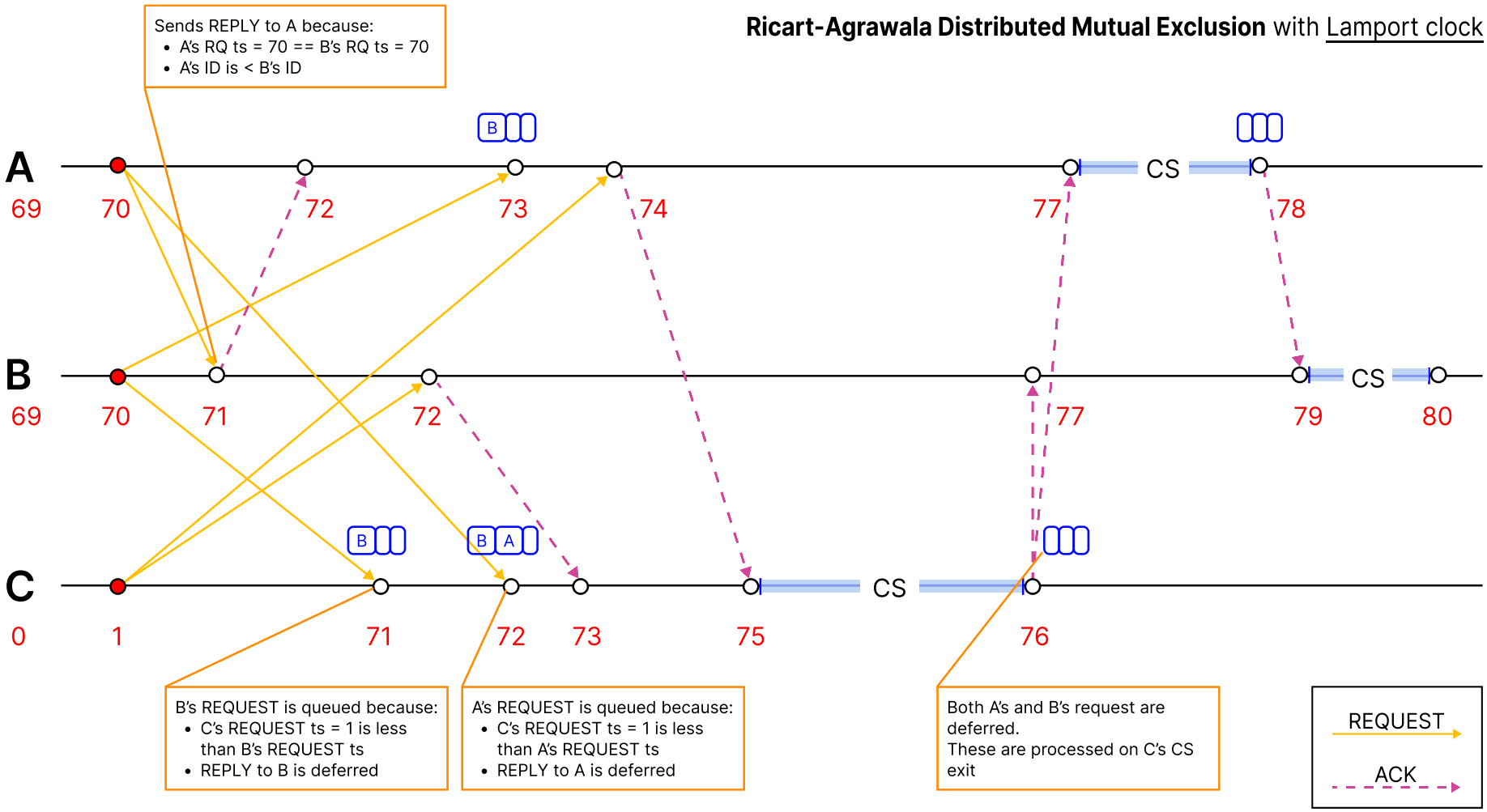


Lamport / Ricart-Agrawala – A comparison side by side

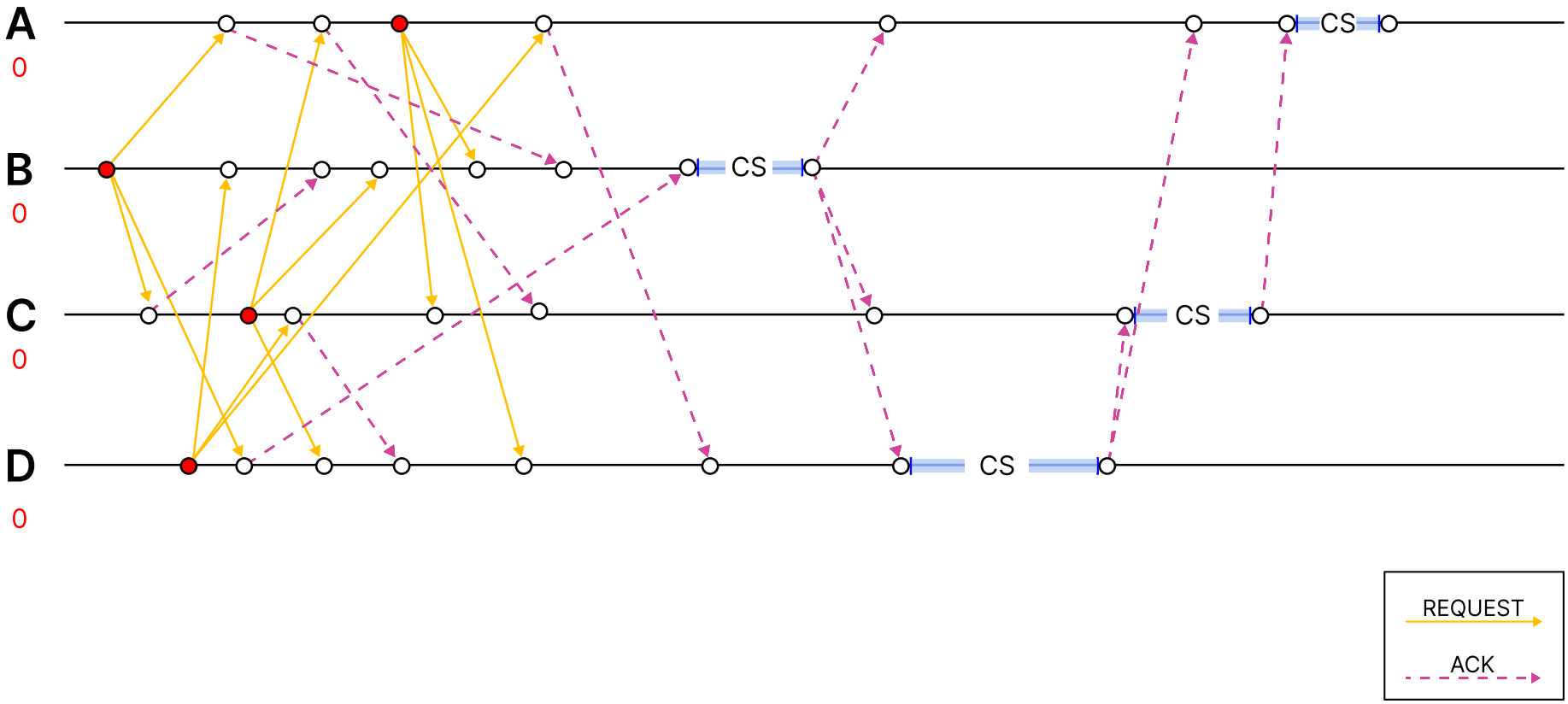
Ricart-Agrawala Distributed Mutual Exclusion with Lamport clock



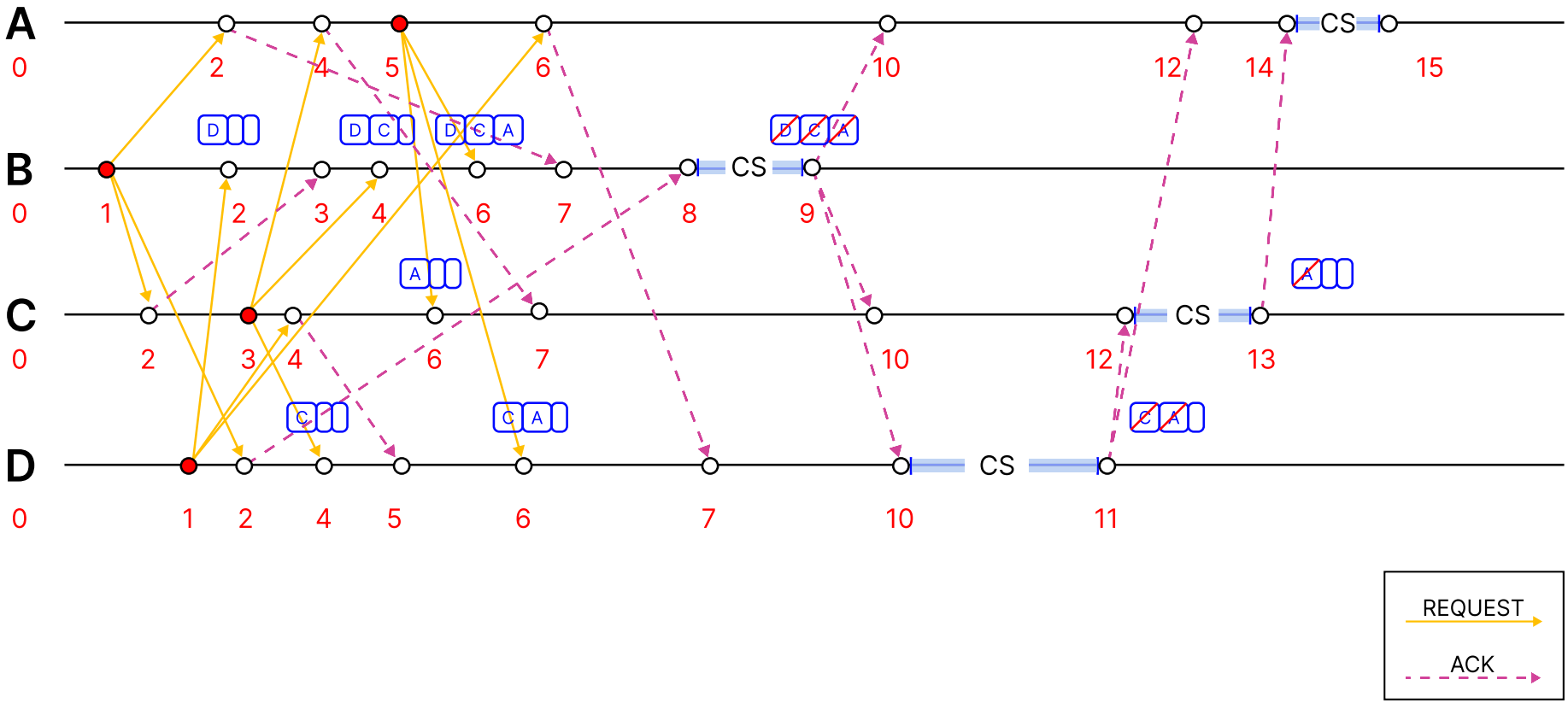
Ricart-Agrawala's algorithm – A more complex example



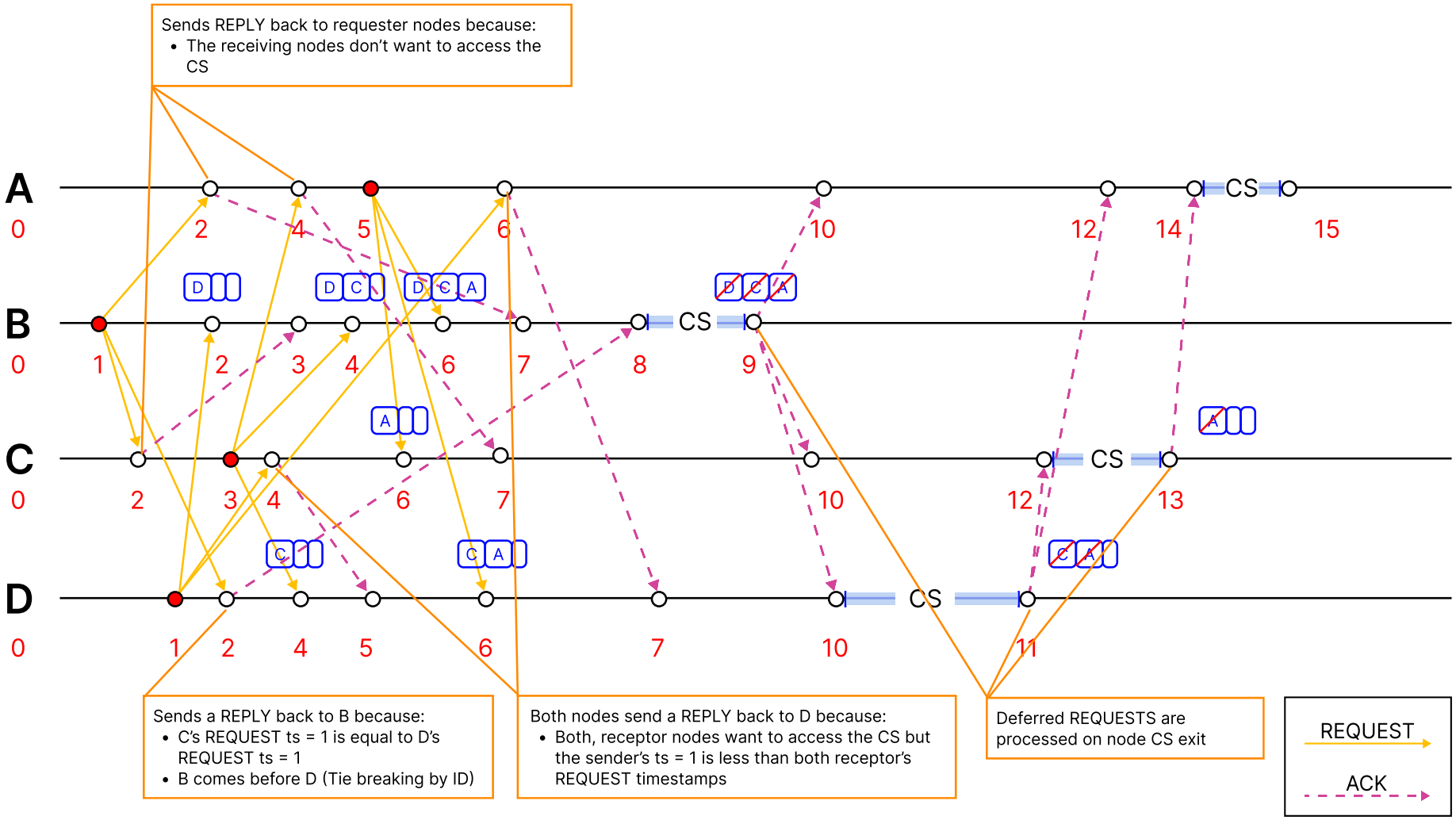
Ricart-Agrawala's distributed mutex exercise



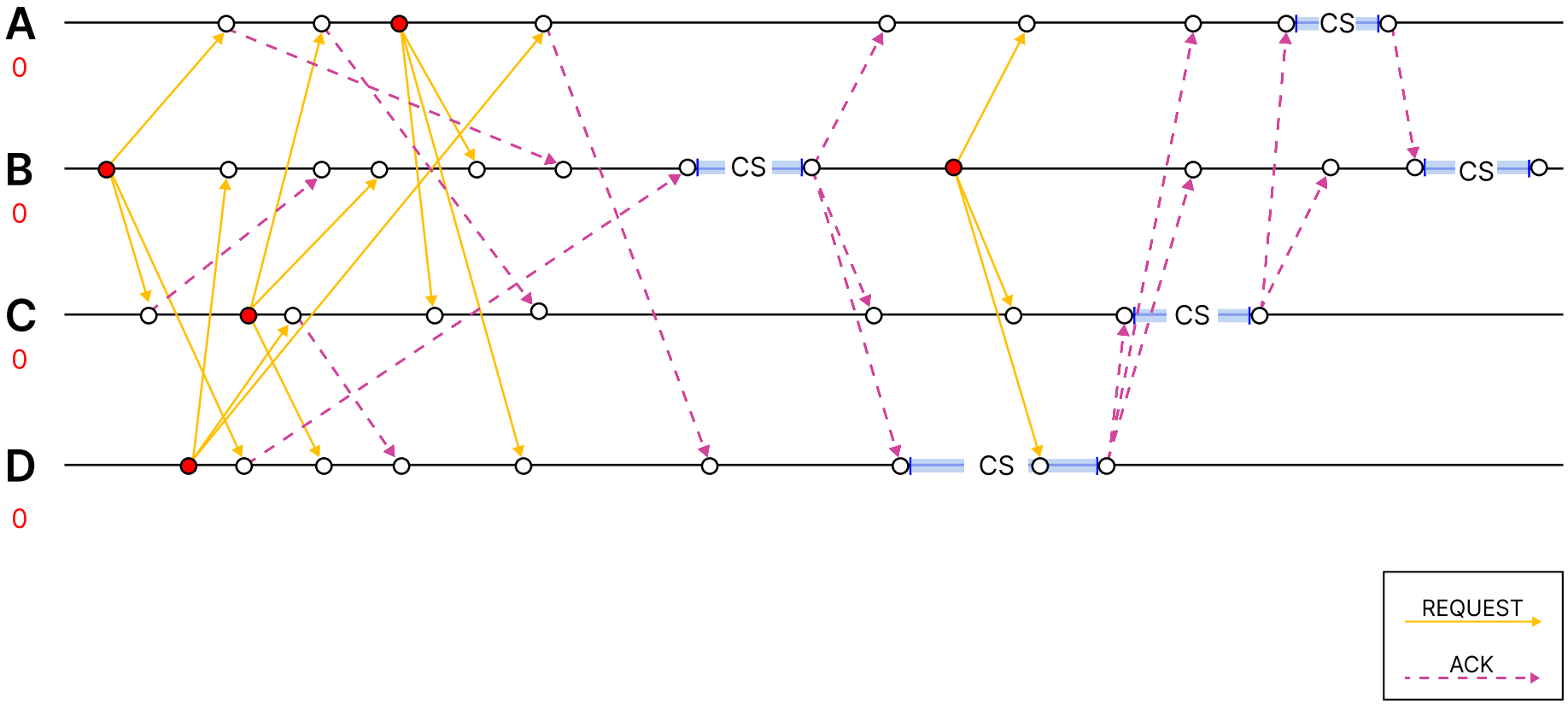
Ricart-Agrawala's distributed mutex exercise



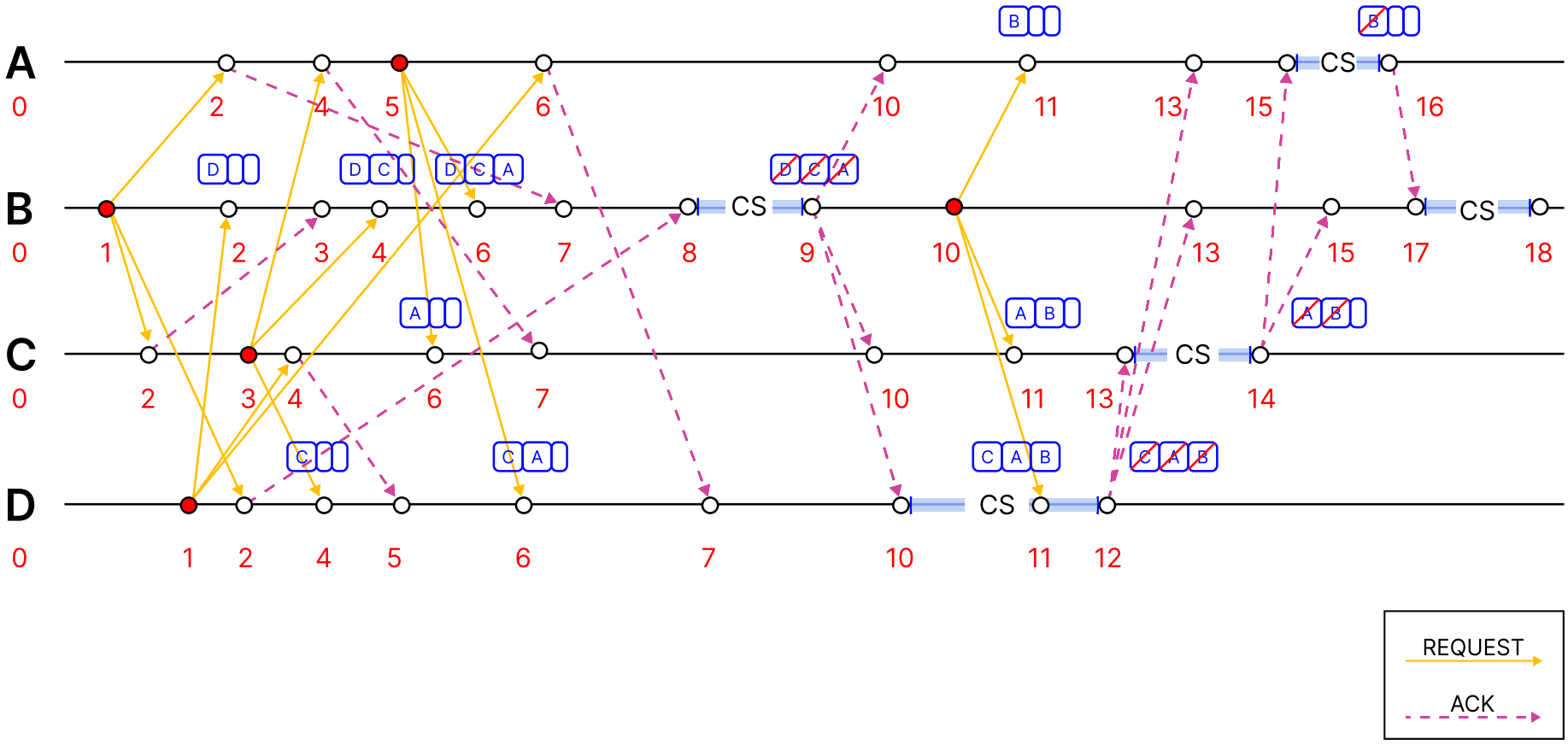
Ricart-Agrawala's distributed mutex exercise



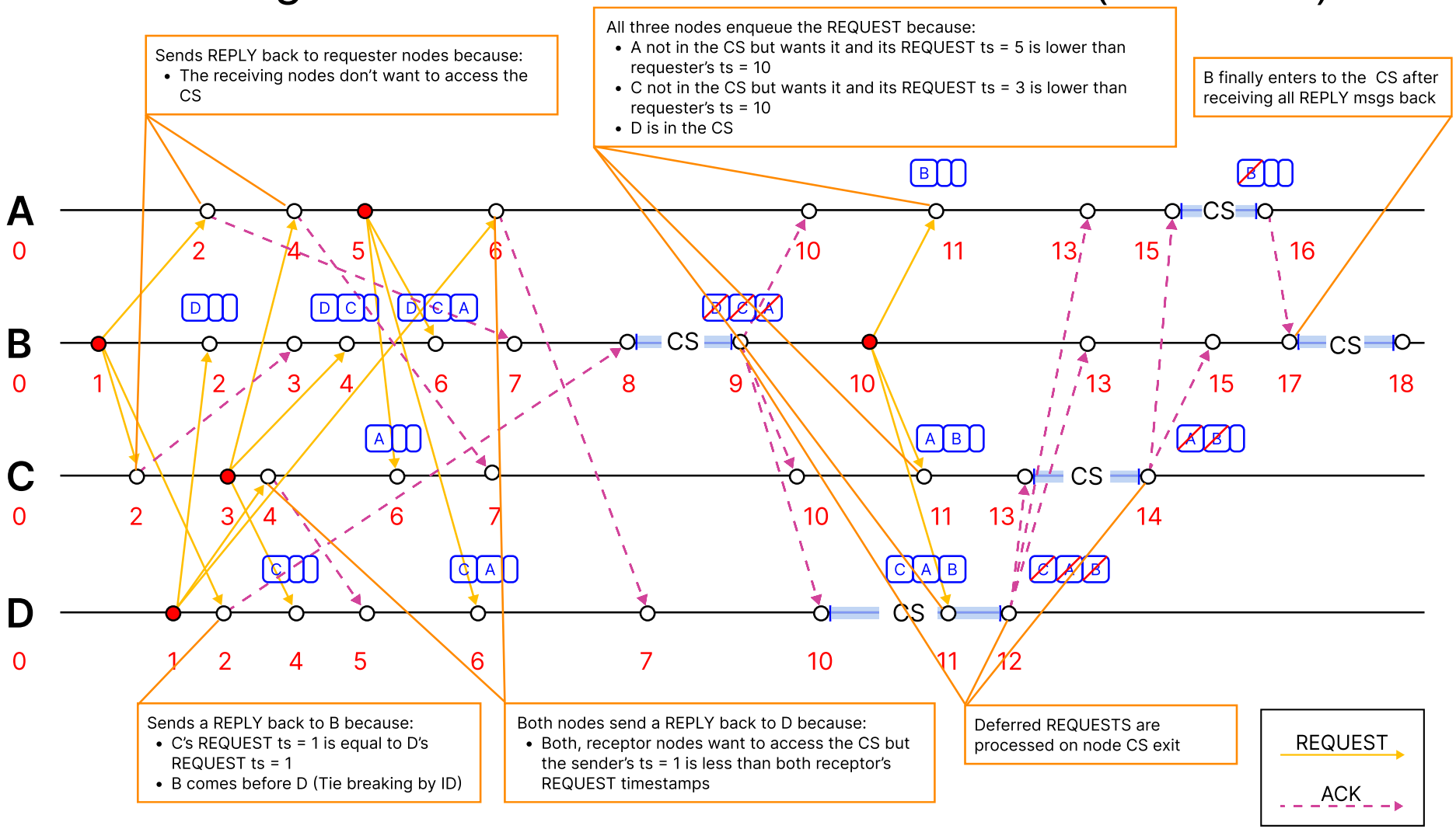
Ricart-Agrawala's distributed mutex exercise (extended)



Ricart-Agrawala's distributed mutex exercise (extended)



Ricart-Agrawala's distributed mutex exercise (extended)



Appendix: Ricart-Agrawala mutex

```
public class RAMutex extends Process implements Lock {
  int myTS;
  LamportClock clock = new LamportClock();
  LinkedList prendingQ = new LinkedList<Integer>();
  int numOkay = 0;

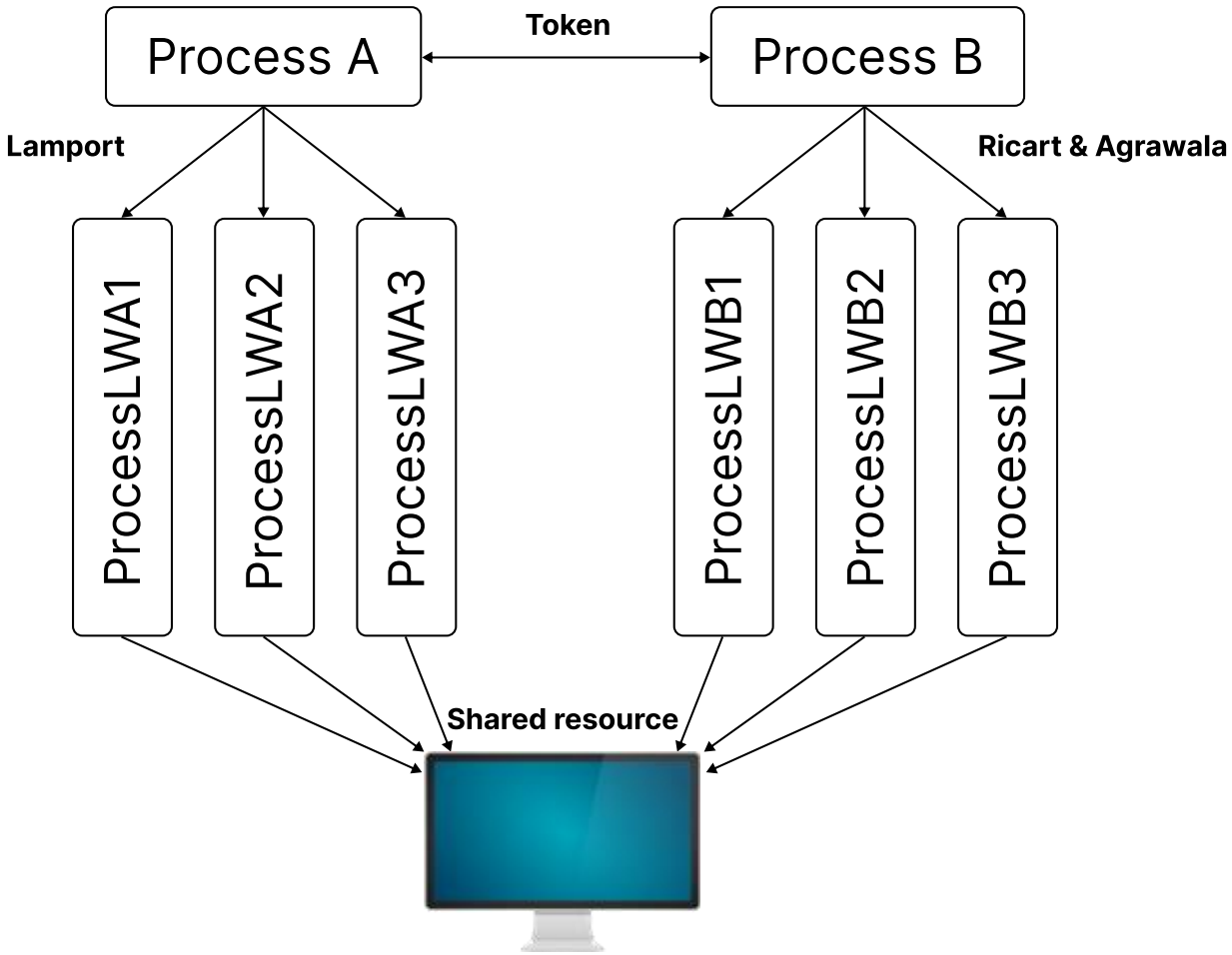
  public RAMutex(Linker initComm) {
    super(initComm);
    myTS = Symbol.Infinity;
  }

  public synchronized void requestCS() {
    clock.tick();   // Increase timer before sending event
    myTS = clock.getValue();
    broadcastMsg("request", myTS);
    while(numOkay < N - 1) wait();
  }

  public synchronized void releaseCS () {
    myTS = Symbol.Infinity;
    clock.tick(); // Increase timer before sending event
    while(!pendingQ.isEmpty()) {
      int pid = pendingQ.removeHead();
      sendMsg(pid, "okay", clock.getValue());
    }
  }

  public syncrhonized void handleMsg(Msg m, int src, String tag) {
    int timestamp = m.getMessageInt();
    clock.receiveAction(src, timeStamp);    // Increase timer on reception event
    if (tag.equals("request")) {
      if ((myTS == Symbols.Infinity)        // Node not interested in CS
          || (timeStamp < myTS)
          || ((timeStamp == myTS) && (src < myID))) {
        sendMsg(src, "okay", clock.getValue());
      } else {
        pendingQ.add(src);
      }
    } else if (tag.equals("okay")) {
      numOkay++;
      if (numOkay == N - 1) notify();   // Unlock wait and enter to CS
    }
  }
}
```

Assignment


Process A
Token
Process B
Lamport
Ricart & Agrawala
ProcessLWA1
ProcessLWA2
ProcessLWA3
ProcessLWB1
ProcessLWB2
ProcessLWB3
Shared resource